UMHACKATHON 2026 · DOMAIN 1 · AI SYSTEMS & AGENTIC WORKFLOW AUTOMATION

Technical Architecture.

Stack, ILMU/GLM integration, the workflow engine, and the trade-offs we made to ship the MVP.

TEAM · OPUS MAGNUM

Oybek Odilov
Al-Bazeli Gameel Abduljalil
Lee Yih Shen
Samandar Erkinjonov
Abdugaffor Odilov

ONBOARD

POWERED BY Z.AI GLM · REGULATED ONBOARDING ENGINE

TAD · TECHNICAL ARCHITECTURE APRIL 2026 SUBMISSION RELEASE

1. Stack at a Glance

Backend

LAYER	CHOICE	PIN	WHY
Reasoning engine	ILMU `ilmu-glm-5.1` via OpenAI SDK	`openai ⩾ 1.54`	Mandatory for the hackathon; full OpenAI surface (tools, JSON mode, streaming)
Backend language	Python	≥ 3.11	First-class LangGraph + Pydantic + OpenAI SDK ecosystem
Web framework	FastAPI + Pydantic v2	`fastapi ⩾ 0.115` / `pydantic ⩾ 2.8`	Typed request/response, SSE via `sse-starlette`, zero boilerplate
Orchestration	LangGraph	`⩾ 0.2.50`	Native stateful graphs, checkpointer, conditional edges, human-in-loop
Checkpointer	`langgraph-checkpoint-sqlite` / `-postgres`	both `⩾ 2.0`	Same interface, dev and prod
Database driver	`psycopg[binary]`	`⩾ 3.2`	Async-friendly, no separate compile step on Render
ORM	SQLAlchemy 2.0	`⩾ 2.0`	JSONB on Postgres / TEXT on SQLite — same code
PDF extraction	`pypdf`	`⩾ 4.3`	Pure Python, text-layer first
OCR fallback	`pytesseract` + `Pillow` (`ocr` extra)	`⩾ 0.3.13` / `⩾ 10.4`	Kicks in when PDFs lack a text layer
SSE	`sse-starlette`	`⩾ 2.1`	One-way token streams without a WebSocket layer

LAYER	CHOICE	PIN	WHY
Multipart upload	`python-multipart`	`⩾0.0.9`	Required by FastAPI for file uploads
HTTP client	`httpx`	`⩾0.27`	Smoke tests, ad-hoc fetches

Frontend

LAYER	CHOICE	PIN	WHY
Framework	Next.js App Router + TypeScript	16.2 / 5.x	Routing, server-rendered directory, static landing
Runtime	React	19.2	Server components on read pages, client on interactive
Styling	Tailwind CSS 4	`^4`	Dark token system; PostCSS plugin keeps zero config
UI primitives	Radix UI	dialog/tabs/dropdown/toast	Headless, accessible; thin wrappers in `components/ui/`
Icons	`lucide-react`	`^1.11`	Single icon vocabulary across surfaces
Markdown render	`react-markdown`	`^10.1`	Used for assistant streams and template explanations
Workflow canvas	`@xyflow/react` (React Flow)	`^12.10`	Custom step nodes per primitive, live status, drag-reorder in template editor
3D	`react-three-fiber` + `@react-three/drei` + `three`	9.6 / 10.7 / 0.184	Real GLB apartment models (CC-BY from Poly Pizza)
Realtime	Server-Sent Events	native	Simpler than WS for one-way token streams
Auth	Mocked session (seeded users)	—	Out of scope; flagged in PRD

Deploy & ops

CONCERN	CHOICE	NOTES
Backend hosting	Render (single Blueprint)	`render.yaml` provisions web service + managed Postgres 16
Frontend hosting	Vercel	Static Next.js build; CORS opens for the Render origin
Local dev DB	SQLite at `var/dev.sqlite`	Same SQLAlchemy code path
Local dev compose	`docker-compose.yml`	Postgres-only service for parity testing
Env loading	`pydantic-settings` + `python-dotenv`	`.env` ungitignored values; `.env.example` ships the variable names

2. GLM / ILMU Integration

2.1 Endpoint and SDK

- **Base URL** `https://api.ilmu.ai/v1`
- **Auth** `Authorization: Bearer $GLM_API_KEY`
- **SDK** `openai.OpenAI(api_key=..., base_url=...)` — ILMU is fully OpenAI-compatible.

The single point of contact with the model is `backend/app/glm/client.py`. `GLMClient` exposes three public surfaces:

- `complete(...)` — free-form chat, optional streaming.
- `complete_json(...)` — JSON-mode with a schema-hint system prompt.
- `call_tool(tool_spec=...)` — forced function call, returns parsed arguments.

All three:

- retry `RateLimitError` / 5xx with exponential backoff and jitter (up to 4 attempts),
- emit a `Decision` to an optional sink (the primitive-level decision log),
- default to `reasoning_effort="low"` on structured-output paths (see §2.3).

2.2 Provider switch — and why GLM is canonical

The OpenAI SDK is the *transport*. The *brain* is `ilmu-glm-5.1` and only `ilmu-glm-5.1`. `GLMClient` reads `AI_MODEL` (default `glm`) and selects between:

- `AI_MODEL=glm` → `GLM_BASE_URL` (default `https://api.ilmu.ai/v1`) + `GLM_MODEL` (default `ilmu-glm-5.1`)
- `AI_MODEL=openai` → `OPENAI_BASE_URL` + `OPENAI_MODEL`

This switch is a **local-development convenience**, not a runtime swap. It exists so an engineer whose ILMU key is rate-limited during a debugging session can point the same client at any OpenAI-compatible endpoint to validate that a tool spec parses, that streaming flows, that retry semantics are sane. **The submission, the seed data, the demo path, and every test fixture run end-to-end on `ilmu-glm-5.1`**; the OpenAI route has not been validated against the seven-tool repertoire and is not in the production envelope. Every persisted `decision_log` entry in the demo database is a GLM decision.

2.3 Capabilities verified empirically

Smoke tests in `backend/scripts/glm_smoke.py`:

CAPABILITY	RESULT	EVIDENCE
Plain chat completion	✓	`smoke_plain` returns expected word
JSON mode (`response_format={"type":"json_object"}`)	✓	`smoke_json` matches schema
Tool use / function calling	✓	`smoke_tool_use` returns correct structured args
Streaming (SSE)	✓	4+ delta chunks observed
`reasoning_effort="low/medium/high"`	✓	Standard param; unlocks thinking budget
Vision / image inputs	✗	Model replies "I cannot see any image" for any PNG payload; OCR fallback is the baseline

All eight product tools were validated against real model output during the build.

2.4 Reasoning-effort discipline

`ilmu-glm-5.1` is a **thinking model**: part of the token budget goes to internal reasoning. Two practical consequences:

- **Small `max_tokens` ceilings empty `content`.** Under 256 tokens is unsafe for free-form output. Our client logs a warning and retries; the production-correct choice is to budget ≥ 512 tokens on visible responses.
- **Structured outputs run well on `reasoning_effort="low"`.** `call_tool` and `complete_json` default to `low` — extraction, verification, cross-check, and autofill don't benefit from deep reasoning and `low` is materially faster.

Free-form assistant streams (visitor chat, template tutor) omit `reasoning_effort` so the model's default applies; the system prompt governs tone and grounding discipline instead.

2.5 Audit trail: `Decision`

Every GLM call produces a `Decision` dataclass:

```
{
  "call_id": "...",
  "tool": "verify_against_criteria",
  "request": {"model": ..., "messages": [...redacted...], "tools": [...]},
  "response": {"content": ..., "tool_calls": [...], "finish_reason": ..., "usage":
  "duration_ms": 842,
  "ts": 1714032384.1,
  "step_id": "ownership"   # tagged by the primitive
}
```

Primitives pass a local sink to `GLMClient(sink=...)` so decisions land tagged with the owning step id. The runner merges these into `step_runs.decision_log` on every tick. Long content fields (over 4 k chars) are elided and any inline `image_url` data URIs are stripped — the log stays scannable and the database doesn't accumulate raw documents.

3. Workflow Engine: Template → StateGraph

3.1 Primitive vocabulary

Six primitives — the full vocabulary for all templates — and the GLM tools each one calls:

PRIMITIVE	GLM TOOLS USED
`collect_form`	optional `explain_field`, optional `propose_auto_fix`
`upload_compliance`	`extract_document`, `verify_against_criteria`, optional `propose_auto_fix`
`upload_content`	`extract_document`
`cross_check`	`cross_check_documents`
`human_review`	`summarize_for_review`
`publish`	— (terminal)

3.2 Compiler

`compile_template(steps)`:

1. Constructs a `StateGraph(RunState)`.
2. For each step, looks up the factory in `REGISTRY` (populated by decorated primitive modules) and adds a node with the step id.
3. For each step, adds a conditional edge keyed by `_route_after_step`: route to `END` when `awaiting_user` is true, otherwise to the next step (or `END` for the last).
4. Sets the entry point and compiles with the passed-in checkpointer.

Conditional routing per primitive (e.g., the `upload_compliance` re-upload loop) is expressed by the primitive's own return values — the compiler is agnostic.

3.3 RunState reducers

`RunState` is a `TypedDict` with reducers on append-only fields:

- `step_outputs` — shallow dict merge (later tick wins per key).
- `uploaded_docs`, `verification_results`, `decision_log` — list concatenation.
- `profile_draft`, `pending_input` — shallow dict merge.
- `rev` — monotonically increasing integer used to allocate fresh checkpoint threads on rewind.

The reducers guarantee that each node return is a *patch*, not a full state replacement — primitives never accidentally erase prior work.

3.4 Human-in-the-loop handshake

LangGraph supports `interrupt()` natively, but we chose a simpler "paused-by-flag" model:

- Primitive returns `awaiting_user=True, awaiting_step_id=<step>` when it needs input.
- `_route_after_step` routes to `END` on that flag.
- The API endpoint submitting the step merges the payload into `pending_input[step_id]` and re-invokes the graph.
- Primitive sees `pending_input[step_id]`, consumes it, and advances.

Advantages: transparent state (the pause is a state flag, not an interrupt token), trivial to test, trivial to resume. Trade-off: one extra `if pending_input[step_id]` check per primitive; worth the clarity.

3.5 Rewind

`POST /buildings/{id}/run/rewind/{stepId}` increments `RunState.rev` and uses `thread_id = "{run_id}:{rev}"` for the checkpoint. A fresh thread means LangGraph starts from a clean checkpoint, but the previous `step_runs` rows survive in the audit table — every rewind is recoverable. This was a deliberate alternative to mutating an existing checkpoint thread, which is fragile under concurrent ticks.

4. Frontend Architecture

4.1 Next.js 16 App Router

- Server components for data-heavy read pages (the public directory).
- Client components for interactive surfaces (workflow canvas, step panel, assistant chat, template editor).
- Dynamic routes with `params: Promise<{...}>` unwrapped via `use(params)` (Next 16 convention).

4.2 API client

`frontend/lib/api.ts` is a small typed fetch wrapper. Types mirror backend Pydantic models by hand — drift is caught on the next compile.

4.3 SSE helper

`frontend/lib/sse.ts` parses `text/event-stream` from POST responses using a manual `fetch` + `ReadableStream` reader. Native `EventSource` is GET-only and not usable for chat-

POST flows.

4.4 React Flow canvas

`@xyflow/react` 12 powers two surfaces:

- **Owner workflow canvas** — custom `StepNode` type with a per-primitive icon, live status badge (`pending / running / awaiting_user / passed / failed`), and a pulse-ring animation on states that need attention. Click routes to the step panel.
- **Admin template editor** — drag-reorder of steps, primitive-typed configuration panels, optimistic save into `workflow_templates.steps` .

A small graceful-degradation detail: the form renderer **falls back to field index when the schema's `name` key is missing** (a real bug we hit on a hand-authored template). A field at index 2 still renders and submits even when the template author forgot to set `name: "address"` .

4.5 3D scene

`react-three-fiber` canvas loading a CC-BY GLB apartment model from `public/models/` . Each building gets a deterministic model via a hash of its id, so repeated visits show the same building but the directory has visual variety. Models are normalised via a bounding-box scale-to-fit so different source scales render consistently. Scene-config stats (floors / units / footprint) are overlaid via drei's `<Html>` . The visitor assistant emits scene-navigation cues (camera moves) rather than asking for photos it doesn't have.

5. Persistence & Storage

5.1 Database

SQLAlchemy 2.0 with `DeclarativeBase` . One session per request (FastAPI `Depends`). JSON columns use `sqlalchemy.JSON` which maps to JSONB on Postgres and TEXT on SQLite — app code uses the same types regardless.

Schema bootstrap via `Base.metadata.create_all()` . Alembic was deliberately deferred for hackathon velocity (acknowledged in the PRD). Reseeding goes through `app.seed.reseed_demo` , which calls `drop_all` / `create_all` on app tables and **clears LangGraph checkpoint rows in place** (rather than unlinking the SQLite file). That keeps an in-flight backend process from holding a stale inode after the file is replaced.

5.2 Object store

`STORAGE_BACKEND=local` writes uploaded files under `var/uploads/<building_id>/<step_id>/<doc_id>__<filename>`. The production path is S3 with the same key layout; only the backend picker needs to change.

5.3 LangGraph checkpointer

- **Dev:** SQLite file at `var/langgraph.sqlite` (`SqliteSaver`).
- **Prod:** Postgres at the same `DATABASE_URL` (`PostgresSaver`).

`runner.py` picks the right saver by reading the URL scheme; callers never touch the checkpointer directly.

6. Key Trade-offs

DECISION	ALTERNATIVE CONSIDERED	RATIONALE
LangGraph over a custom state machine	Hand-rolled Python generator	Built-in checkpointer, human-in-loop semantics, conditional edges, ecosystem
Template as JSON in DB	YAML files in repo	Admin UX: the admin *is* the author at runtime; file-system templates would require a redeploy per city
Two doc classes (compliance vs content)	One class with a role tag	Clearer UI contract; different verification vs extraction flows; "never shown publicly" vs "shown publicly" is a user-facing invariant
SSE over WebSocket	WebSocket	One-way streams suffice; SSE is trivially resumable and CORS-friendly
`create_all` over Alembic	Alembic	Schema is small; any change in the hackathon window is a full reset. Alembic is the production upgrade path
Reseed clears tables in place	Unlink the SQLite file	Avoids stale inode in the running backend process
Real GLB apartment models	Extruded-box placeholders	Visual payoff matters for pitch judging; CC-BY models from Poly Pizza cost nothing and look like real buildings
Fresh-compile per tick	Compile-once cache	The cache held stale checkpointer references across requests. Compile is cheap (microseconds); correctness first
OpenAI SDK over raw HTTP	`httpx` + hand-rolled retry	Retry semantics, streaming iterator, structured-output typing already in the SDK
Rewind via `RunState.rev` + new thread	Mutate the existing checkpoint thread	Fresh thread is atomic; the prior thread stays inspectable; no concurrent-tick collisions
Autofill as a separate `propose_auto_fix` tool	Reuse `extract_document` arguments	The two tasks have different system prompts (extract vs reconcile); two tools keep prompts narrow and audit logs legible

7. Dev Experience

```
# one-time
just install           # venv + pip + pnpm install
cp .env.example .env   # then set GLM_API_KEY

# daily
just db-up             # Postgres via docker compose (or skip for SQLite)
just seed              # admin, owner, Shah Alam, template, Menara Demo
just backend-dev       # uvicorn on :8000
just frontend-dev      # next dev on :3000

# quality
just test              # pytest (unit + integration; live-GLM marked skipif)
just smoke-glm         # capability matrix in §2.3
```

A Makefile mirrors the justfile for environments without `just`.

8. Deploy

A single Render Blueprint (`render.yaml`) provisions:

- a Python web service running `uvicorn app.main:app` ,
- a managed Postgres 16 instance,
- environment wiring for `DATABASE_URL` , `GLM_API_KEY` , `STORAGE_BACKEND` , and CORS origins.

Recent fixes worth noting: corepack's pnpm signing-key verification was breaking the Render build, so the Blueprint installs pnpm explicitly; the pnpm workspace file was removed because the frontend is a single package.

The frontend deploys to Vercel as a standard Next.js project. Both halves are stateless and horizontally scalable — the only stateful component is Postgres.

9. Known Limitations

- No vision path on `ilmu-glm-5.1` (empirically confirmed). Images degrade to OCR text.
- Single-tenant demo: mocked auth, no per-user isolation beyond the seeded ids.
- SSE endpoint uses a 500 ms poll of the DB. Fine for one concurrent demo run; a pub/sub bus (Redis) is the upgrade path.
- No embeddings / RAG. Visitor assistant grounds on the structured profile + raw content-doc text pulled in-context (GLM has 200 k context; single-building content is well within budget).

- The `AI_MODEL=openai` route is a development convenience and is not part of the submission envelope.

10. Security Notes (within the MVP scope)

- `.env` (with `GLM_API_KEY`) is gitignored. `.env.example` ships the variable names only.
- FastAPI CORS allows only the configured frontend origin (localhost in dev, the Vercel domain in prod).
- Uploaded files are written under `var/uploads/…`; filenames are sanitised to a basename before writing.
- No SQL string concatenation — all queries are SQLAlchemy ORM.
- Decision logs redact message bodies over 4 k chars and elide `image_url` content — keeps the log readable and prevents the database from caching whole documents.

See the SAD for data flows and module boundaries; the QATD for test strategy and risk register.